VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



HỌC MÁY – CO3117

BÁO CÁO BÀI TẬP LỚN

PHÂN LOẠI CHỦ ĐỀ TIN TỨC TRÊN BỘ DỮ LIỆU AG NEWS

Giảng viên hướng dẫn: VÕ THANH HÙNG

Nhóm học tập: 20 – Lớp L03

Danh sách thực hiện:

1. Nguyễn Mai Việt Hòa – 2411141
2. Ngô Tuấn Khôi – 2411697
3. Nguyễn Đăng Gia Dạo – 2410652

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 11 NĂM 2026

Mục lục

Danh mục hình ảnh **3**

Danh mục bảng **4**

Tóm tắt **5**

Phân công công việc **6**

1. Tổng quan bài toán và dữ liệu **7**

a. Nguồn dữ liệu 7

2. Chuẩn bị dữ liệu và phân tích đặc trưng **9**

a. Tạo văn bản đầu vào 9

b. Thống kê độ dài văn bản 10

c. Từ phổ biến và từ đặc trưng theo lớp 12

3. Pipeline học máy truyền thống **13**

a. Tiền xử lý dữ liệu 14

b. Trích xuất đặc trưng 15

c. Huấn luyện và đánh giá mô hình 17

d. So sánh trực quan và phân tích kết quả 23

e. Đánh giá tập test và phân tích lỗi theo lớp 25

f. Thực nghiệm lựa chọn đặc trưng (Feature Selection) 28

4. Tổ chức và quản lý mã nguồn của nhóm **30**

a. Cấu trúc thư mục 30

b. Luồng chạy tái lập 32

5. Thảo luận **33**

6. Kết luận và hướng phát triển **34**

Danh mục hình ảnh

1 Phân phối nhãn của tập huấn luyện (trái) và tập kiểm tra (phải). Hai biểu đồ dùng thang đo trục tung khác nhau. 9

2 Phân phối độ dài văn bản theo số ký tự (trái) và số từ (phải). Trục tung biểu thị số mẫu trong mỗi khoảng; phần lớn mẫu tập trung ở vùng độ dài ngắn, với đuôi phân phối kéo dài về bên phải. 11

3 Số từ theo từng chủ đề trên tập huấn luyện. Điểm ngoại lai không được vẽ để làm rõ vùng trung tâm. Trung vị lần lượt là **39** từ (World), **37** từ (Sports), **37** từ (Business) và **36** từ (Sci/Tech). Do đó, độ dài không phải tín hiệu phân biệt mạnh. 11

4 Mười lăm token chữ cái xuất hiện nhiều nhất trên văn bản đã ghép, trước khi loại stopword. 12

5 Mười token xuất hiện nhiều nhất trong từng lớp sau khi bỏ stopword. Bước lemmatization chưa được áp dụng trên hình này. 13

6 Chuỗi tiền xử lý văn bản của pipeline học máy truyền thống. 15

7 Accuracy của ba thuật toán trên BoW và TF–IDF (validation). Trục tung được thu hẹp quanh vùng 0,87–0,92 để thấy chênh lệch giữa các cấu hình. 23

8 Macro-F1 của cùng sáu cấu hình trên tập validation. Thứ hạng gần như trùng với accuracy vì bốn lớp có cùng số mẫu. 24

9 Thời gian huấn luyện trên tập validation. Naive Bayes gần như tức thời. Linear SVM trên BoW là cấu hình chậm nhất; cùng thuật toán trên TF–IDF nhanh hơn rõ rệt. 24

10 Ma trận nhầm lẫn của Linear SVM + TF–IDF trên 7.600 mẫu test. Hàng là nhãn đúng, cột là nhãn dự đoán. 26

11 Precision, recall và F1 theo lớp trên tập test (Linear SVM + TF–IDF). Trục tung bắt đầu ở 0,80 để thấy Sports tách khỏi hai lớp kinh doanh và công nghệ. 27

12 Mười hệ số dương lớn nhất của Linear SVM trên từng lớp. Trục hoành là trọng số trong không gian TF–IDF. 28

13 Accuracy và macro-F1 tăng khi giữ nhiều đặc trưng chi-bình phương hơn. Tại $k = 500$, cả hai độ đo còn khoảng 0,86. 30

Danh mục bảng

1 Phân công chi tiết nội dung báo cáo và mã nguồn lập trình 6
2 Phân phối nhãn trên hai tập dữ liệu 8
3 Thống kê độ dài văn bản trên tập huấn luyện trước tiền xử lý 10
4 Kích thước các ma trận đặc trưng thực nghiệm 17
5 Kết quả sáu tổ hợp đặc trưng và mô hình trên 18.000 mẫu validation . . 22
6 Chỉ số theo lớp của Linear SVM + TF–IDF trên tập test 26
7 Hiệu năng Linear SVM sau khi chọn đặc trưng bằng Chi-bình phương . . 29

Tóm tắt

Báo cáo trình bày quy trình xây dựng và đánh giá một pipeline học máy có giám sát nhằm phân loại bản tin ngắn thành bốn chủ đề: Thế giới (World), Thể thao (Sports), Kinh doanh (Business) và Khoa học/Công nghệ (Sci/Tech) trên bộ dữ liệu AG News. Bộ dữ liệu cân bằng gồm **120.000** mẫu huấn luyện và **7.600** mẫu kiểm tra. Nhóm chia tiếp tập huấn luyện thành 85% train và 15% validation (**102.000/18.000** mẫu) để so sánh mô hình trước khi đánh giá lần cuối trên tập test. Nhóm áp dụng quy trình xử lý ngôn ngữ tự nhiên (NLP) để làm sạch văn bản, xóa URL, tách từ, loại bỏ stopword và chuẩn hóa từ. Sau đó, nhóm trích xuất đặc trưng bằng Bag-of-Words (BoW) và TF–IDF với không gian vector tối đa **5.000** từ.

Để đánh giá hiệu năng, nhóm huấn luyện sáu tổ hợp mô hình dựa trên ba thuật toán truyền thống: Multinomial Naive Bayes, Logistic Regression và Linear SVM.

Kết quả trên tập validation cho thấy mọi mô hình đều vượt xa đường cơ sở dự đoán ngẫu nhiên. Tổ hợp Linear SVM kết hợp TF–IDF đạt hiệu năng cao nhất, với độ chính xác (accuracy) **0,9138** và macro-F1 **0,9136**. Thời gian huấn luyện của cấu hình này khoảng **2,79 giây**. Trên tập test độc lập, cùng cấu hình này đạt accuracy **0,9051** và macro-F1 **0,9049**.

Phân tích lỗi cho thấy chủ đề Thể thao có độ phân biệt rõ nhất, trong khi phần lớn lỗi phân loại xuất phát từ sự chồng lấn từ vựng giữa Kinh doanh và Khoa học/Công nghệ. Thử nghiệm bổ sung về lựa chọn đặc trưng bằng kiểm định Chi-bình phương (χ^2) cho thấy việc giữ trọn vẹn 5.000 từ vựng mang lại độ chính xác cao nhất (0,9138), hạn chế việc đánh mất các từ vựng đặc trưng theo từng chủ đề.

Phân công công việc

Nhóm phân công công việc cụ thể và cân bằng giữa ba thành viên, đảm bảo mỗi thành viên đều tham gia đầy đủ cả phần nghiên cứu lý thuyết, biên soạn báo cáo học thuật và trực tiếp lập trình, kiểm thử các module mã nguồn machine learning của dự án. Tỷ lệ hoàn thành công việc của từng thành viên đạt 100% theo đánh giá và xác nhận chung của toàn nhóm.

Bảng 1. Phân công chi tiết nội dung báo cáo và mã nguồn lập trình

Họ tên	**MSSV**	**Nội dung báo cáo**	**Module mã nguồn đảm nhiệm**	**Tiến độ**
Nguyễn Mai Việt Hòa *(Nhóm trưởng)*	2411141	Tổng quan bài toán, Nguồn dữ liệu & Phân tích khám phá (EDA); Đánh giá kiểm thử tập Test; Tổ chức mã nguồn	• `src/data_loader.py` • `notebooks/01_data_analysis.ipynb` • `notebooks/04_final_test_evaluation.ipynb` • `Dockerfile`, `requirements.txt`, `README.md`	100%
Ngô Tuấn Khôi	2411697	Chuẩn bị dữ liệu & Tiền xử lý; Biểu diễn đặc trưng (BoW, TF–IDF); Thí nghiệm Lựa chọn đặc trưng (Feature Selection); Kết luận & Hướng phát triển	• `src/preprocessing.py` • `src/utils.py` • `notebooks/02_preprocessing.ipynb` • `experiments/feature_selection.py` (χ^2, Mutual Info, Tree-based)	100%
Nguyễn Đăng Gia Đạo	2410652	Cơ sở toán học các mô hình ML; Thiết kế thực nghiệm & Huấn luyện (Validation); Thảo luận & Phân tích lỗi	• `src/models/baseline.py` • `src/models/traditional_ml.py` • `src/evaluation.py` • `notebooks/03_model_training.ipynb`	100%

1. Tổng quan bài toán và dữ liệu

Bài toán của báo cáo là phân loại văn bản có giám sát. Với mỗi bản tin, mô hình dự đoán một trong bốn chủ đề. Nhóm sử dụng AG News – bộ dữ liệu gồm các bản tin ngắn đã có nhãn cùng phép chia sẵn tập huấn luyện và tập kiểm tra. Đặc điểm dữ liệu đặt ra hai yêu cầu khi đánh giá: nhóm cần xử lý các biến thể và nhiễu trong văn bản, đồng thời phải xem xét sự chồng lấn từ vựng giữa một số chủ đề. Ngoài ra, các biểu diễn túi từ được dùng trong báo cáo không mã hóa thứ tự từ, nên chúng không giữ lại toàn bộ ngữ cảnh của câu.

AG News gồm bốn chủ đề: World (Thế giới), Sports (Thể thao), Business (Kinh doanh) và Sci/Tech (Khoa học/Công nghệ). Nhóm giữ nguyên phép chia chính thức gồm 120.000 mẫu huấn luyện và 7.600 mẫu kiểm tra (khoảng 94% và 6% tổng số mẫu). Để chọn mô hình và tránh tối ưu trực tiếp trên tập test, nhóm chia thêm tập huấn luyện thành 85% train và 15% validation bằng `train_test_split` với `stratify` và `random_state=42`, tương ứng 102.000 và 18.000 mẫu. Mỗi bản ghi có ba trường: `class` (nhãn từ 1 đến 4), `title` (tiêu đề) và `description` (mô tả). Hai trường văn bản được dùng làm nội dung đầu vào ở các bước tiếp theo.

Ví dụ bản ghi. Bảng dưới đây minh họa hai mẫu dữ liệu theo các trường `class`, `title` và `description`.

class	title	description
1	Millions Wait Hours in Venezuela to Vote in Recall Election	The unprecedented vote was sluggish as huge crowds lined up at voting booths. Results were not expected until 8 p.m., or later.
2	Leonard Forges Two Clear with Nine to Play at PGA	KOHLER, Wisconsin (Reuters) - American Justin Leonard stayed on track for his second major title in the U.S. PGA Championship final round on Sunday, moving two shots clear with nine holes to play.

a. Nguồn dữ liệu

Nhóm lấy dữ liệu từ trang *AG News Classification Dataset* trên nền tảng Kaggle. Thông tin nhận diện nguồn và thời điểm truy cập được liệt kê dưới đây.

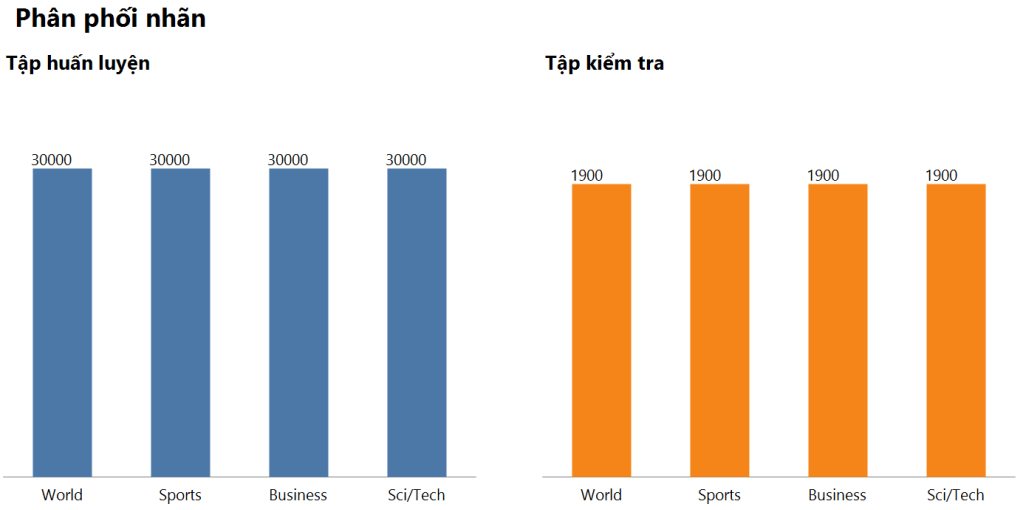
- **Nền tảng cung cấp:** Kaggle.
- **Tên bộ dữ liệu:** AG News Classification Dataset.
- **Đường dẫn truy cập:** https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset.
- **Ngày truy cập:** 21/09/2026.

Phân phối nhãn trong hai tập dữ liệu được trình bày ở Bảng 2. Các lớp có cùng số lượng mẫu trong từng tập.

Bảng 2. Phân phối nhãn trên hai tập dữ liệu

Nhãn	Chủ đề	Huấn luyện	Kiểm tra	Tỉ lệ mỗi tập
1	World	30.000	1.900	25%
2	Sports	30.000	1.900	25%
3	Business	30.000	1.900	25%
4	Sci/Tech	30.000	1.900	25%
Tổng		**120.000**	**7.600**	**100%**

Ghi chú: Tỉ lệ được tính riêng trên từng tập dữ liệu. Mỗi lớp chiếm đúng một phần tư số mẫu.



Hình 1. Phân phối nhãn của tập huấn luyện (trái) và tập kiểm tra (phải). Hai biểu đồ dùng thang đo trục tung khác nhau.

Hình 1 cho thấy số mẫu của bốn lớp bằng nhau trong cả tập huấn luyện và tập kiểm tra. Do mỗi lớp chiếm một phần tư số mẫu, phân phối nhãn thực nghiệm có dạng:

$$\widehat{P}(Y = k) = \frac{1}{4}, \qquad k \in \{1, 2, 3, 4\}. \tag{1}$$

Với phân phối này, nhóm không cần lấy mẫu lại hoặc điều chỉnh trọng số để khắc phục mất cân bằng lớp. Tuy nhiên, sự cân bằng về số lượng không hàm ý rằng các chủ đề có độ khó phân loại tương đương. Vì vậy, phần phân tích tiếp theo tập trung thêm vào đặc điểm của văn bản.

2. Chuẩn bị dữ liệu và phân tích đặc trưng

a. Tạo văn bản đầu vào

Trong bước chuẩn bị dữ liệu, hàm `prepare_dataset()` tạo trường `text` bằng cách nối nội dung `title` với `description`, theo thứ tự này và ngăn cách bằng một dấu cách. Tiếp đó, hàm `add_class_names()` ánh xạ giá trị nhãn số trong `class` (từ 1 đến 4) sang tên chủ đề tương ứng và lưu vào trường `class_name`.

Sau hai thao tác này, dữ liệu có năm cột. Trường `text` chứa nội dung đầu vào để tiền xử

lý và trích xuất đặc trưng. Trường `class` là nhãn đích. Trường `class_name` chỉ dùng để hiển thị và diễn giải, và không được đưa vào mô hình.

b. Thống kê độ dài văn bản

Hai đại lượng được khảo sát trên văn bản đã ghép, trước khi làm sạch: `text_length` (số ký tự) và `word_count` (số từ). Bảng 3 trình bày đầy đủ thống kê của **120.000** mẫu huấn luyện.

Bảng 3. Thống kê độ dài văn bản trên tập huấn luyện trước tiền xử lý

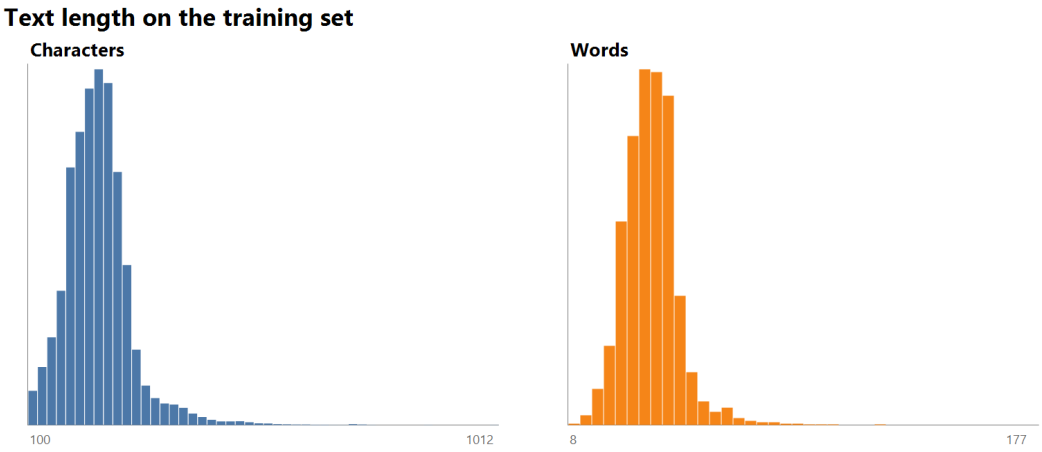
Thống kê	**Số ký tự** `text_length`	**Số từ** `word_count`
Số mẫu (count)	120.000	120.000
Trung bình (mean)	236,460025	37,844617
Độ lệch chuẩn (std)	66,529799	10,088702
Nhỏ nhất (min)	17	4
Phân vị 25% (Q1)	196	32
Trung vị 50%	232	37
Phân vị 75% (Q3)	266	43
Lớn nhất (max)	1.012	177

Ghi chú: Các thống kê được tính trên 120.000 văn bản huấn luyện sau khi ghép tiêu đề và mô tả, trước bước làm sạch.

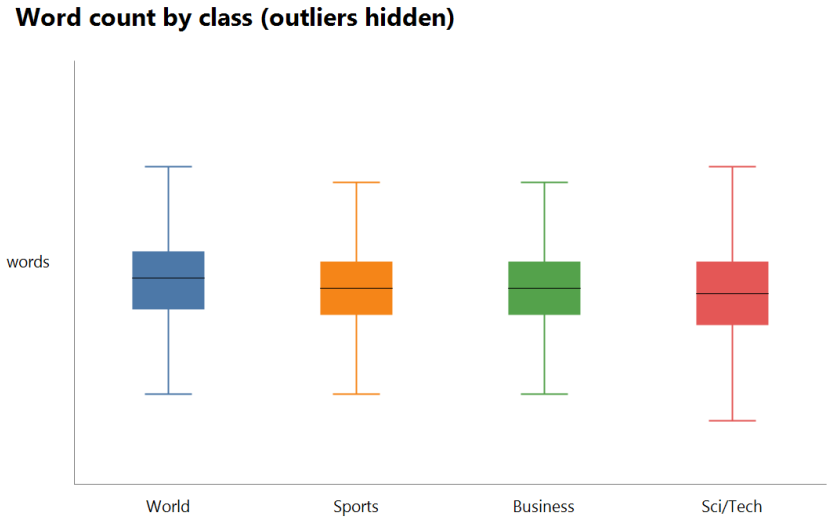
Trung bình mỗi văn bản có khoảng **236,46** ký tự và **37,84** từ, với độ lệch chuẩn lần lượt là 66,53 ký tự và 10,09 từ. Số từ trong bảng là tổng số lần xuất hiện của các từ. Đại lượng này khác với số từ duy nhất (unique words), vì mỗi từ chỉ được đếm một lần trong một văn bản khi tính số từ duy nhất.

Hình 2 cho thấy phần lớn mẫu có độ dài ngắn, tương đương đoạn dẫn hoặc tóm tắt tin tức. Khoảng tứ phân vị có độ rộng $\text{IQR} = Q3 - Q1$ bằng **70** ký tự và **11** từ. Kết quả này cho thấy độ dài tập trung tương đối hẹp ở vùng trung tâm. Trung bình gần trung vị và hơi lớn hơn trung vị. Kết hợp với giá trị cực đại cao, các thống kê này gợi ý phân phối có đuôi phải do một số văn bản dài bất thường. Tuy nhiên, các thống kê trên chưa đủ để định lượng mức độ lệch.

Các mẫu rất ngắn có thể xuất hiện vì tiêu đề hoặc mô tả bị cắt, hoặc vì nhiễu trong dữ liệu. Nhóm cần kiểm tra trực tiếp các mẫu này trước khi kết luận. Nhóm không loại mẫu chỉ dựa vào độ dài nhỏ nhất hoặc lớn nhất.



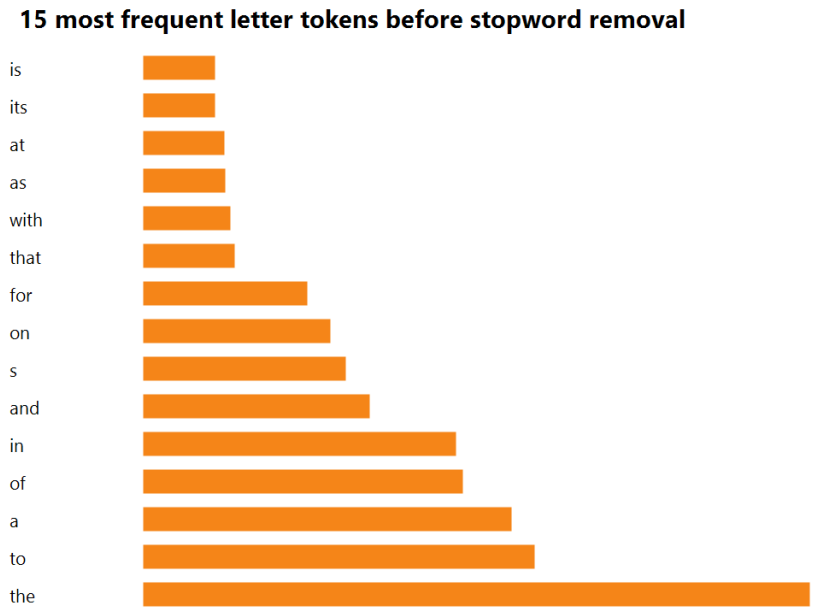
Hình 2. Phân phối độ dài văn bản theo số ký tự (trái) và số từ (phải). Trục tung biểu thị số mẫu trong mỗi khoảng; phần lớn mẫu tập trung ở vùng độ dài ngắn, với đuôi phân phối kéo dài về bên phải.



Hình 3. Số từ theo từng chủ đề trên tập huấn luyện. Điểm ngoại lai không được vẽ để làm rõ vùng trung tâm. Trung vị lần lượt là **39** từ (World), **37** từ (Sports), **37** từ (Business) và **36** từ (Sci/Tech). Do đó, độ dài không phải tín hiệu phân biệt mạnh.

c. Từ phổ biến và từ đặc trưng theo lớp

Trước khi loại stopword, các token chữ cái xuất hiện nhiều nhất là từ chức năng. Trên tập huấn luyện, từ *the* xuất hiện **205.483** lần. Các từ tiếp theo là *to*, *a*, *of*, *in* và *and*. Token *s* cũng đứng cao vì dấu sở hữu cách bị tách thành một chữ cái riêng. Hình 4 giải thích vì sao lớp `TextPreprocessor` phải loại stopword: nếu nhóm giữ các từ này, biểu diễn Bag-of-Words sẽ bị chi phối bởi những từ xuất hiện ở mọi chủ đề.

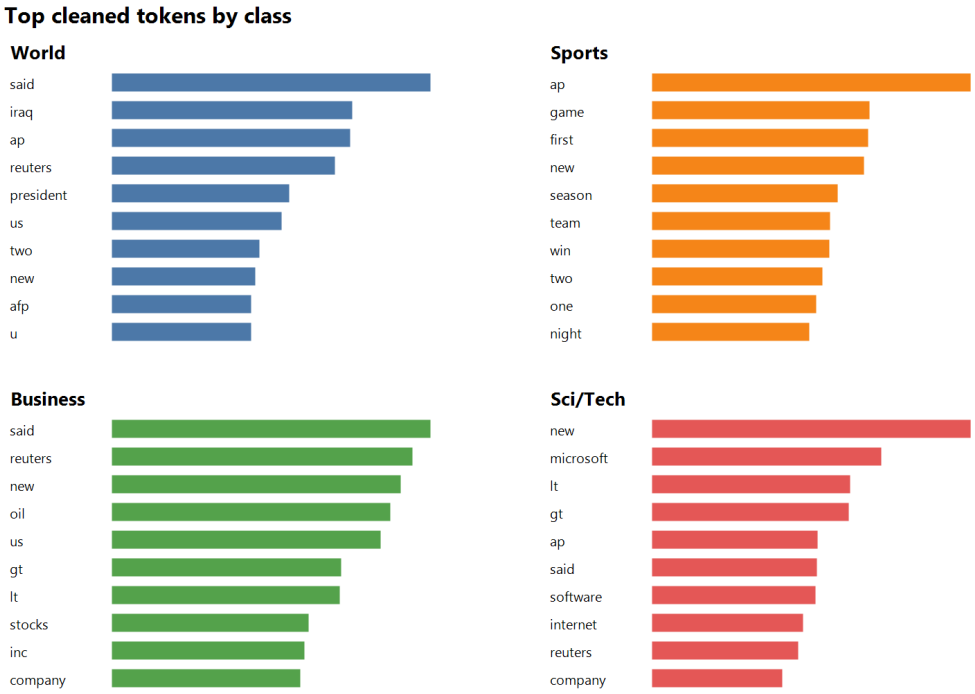


Hình 4. Mười lăm token chữ cái xuất hiện nhiều nhất trên văn bản đã ghép, trước khi loại stopword.

Hình 5 trình bày các từ xuất hiện nhiều nhất trong từng lớp sau khi văn bản được chuyển về chữ thường, loại URL, lọc để chỉ giữ chữ cái và bỏ stopword tiếng Anh. Với lớp World, các từ nổi bật là ***iraq*** và *president*. Với lớp Sports, các từ nổi bật là ***game***, *season*, *team* và *win*. Với lớp Business, các từ nổi bật là ***oil***, *stocks*, *inc* và ***company***. Với lớp Sci/Tech, các từ thường gặp là ***microsoft***, *software* và *internet*. Từ ***company*** xuất hiện trong cả Business và Sci/Tech, nên hai lớp có một phần từ vựng chung. Sự chồng lấn này có thể làm tăng khả năng mô hình nhầm lẫn giữa hai lớp.

Sau các bước lọc, một số token không phản ánh chủ đề vẫn còn trong văn bản. Các token *reuters*, *ap* và *afp* là tên hãng tin. Các token *gt* và *lt* phát sinh từ thực thể HTML `>`

và <. Khi dấu câu và ký tự đặc biệt bị loại bỏ, phần chữ trong thực thể vẫn còn lại. Tương tự, token dwindlingband trong ví dụ tiền xử lý hình thành khi dấu gạch chéo ngược bị xóa mà không được thay bằng khoảng trắng. Đây là các dạng nhiễu từ dữ liệu thu thập, và pipeline hiện tại chưa xử lý riêng các trường hợp này.



Hình 5. Mười token xuất hiện nhiều nhất trong từng lớp sau khi bỏ stopword. Bước lemmatization chưa được áp dụng trên hình này.

3. Pipeline học máy truyền thống

Tập gốc gồm 120.000 mẫu huấn luyện và 7.600 mẫu kiểm tra, được chia đều cho bốn lớp World, Sports, Business và Sci/Tech. Nhãn gốc nhận giá trị trong $\{1, 2, 3, 4\}$. Sau khi tiền xử lý, nhóm chia tập huấn luyện thành 102.000 mẫu train và 18.000 mẫu validation. Trường đầu vào được tạo theo công thức:

$$\texttt{text} = \texttt{title} + \texttt{" "} + \texttt{description}. \tag{2}$$

Việc giữ phép chia chính thức train/test giúp kết quả có thể đối chiếu với các nghiên cứu

dùng AG News. Tập validation chỉ được lấy từ phần train. Bộ vector hóa, trọng số IDF và siêu tham số không được học từ tập kiểm tra.

a. Tiền xử lý dữ liệu

Văn bản tin tức thô chứa chữ hoa/thường không thống nhất, URL, ký tự đặc biệt, số liệu và các token ít mang tín hiệu chủ đề. Lớp `TextPreprocessor` đóng gói một chuỗi xử lý xác định và áp dụng chuỗi xử lý này giống nhau cho tập huấn luyện lẫn tập kiểm tra:

1. **Kết hợp tiêu đề và mô tả.** Tiêu đề thường chứa từ khóa chủ đề, còn mô tả bổ sung ngữ cảnh. Văn bản sau khi nối có trung bình khoảng 236 ký tự và 38 từ.
2. **Chuẩn hóa chữ thường.** Các biến thể như *Reuters* và *reuters* được đưa về cùng một token, tránh làm loãng thống kê.
3. **Loại URL.** Biểu thức chính quy `http\\S+|www\\S+` xóa đường dẫn, vốn thường tạo nhiều token hiếm và ít hữu ích cho phân loại chủ đề.
4. **Chỉ giữ chữ cái tiếng Anh.** Mẫu `[^a-zA-Z\\s]` loại chữ số, dấu câu, ký hiệu tiền tệ và dấu gạch ngang. Quyết định này giảm từ vựng hiếm nhưng đồng thời có thể làm mất tín hiệu như giá cổ phiếu hoặc tỷ số thể thao.
5. **Chuẩn hóa khoảng trắng.** Lệnh tương đương `re.sub(r'\\s+', ' ', text)` gộp các khoảng trắng liên tiếp và loại khoảng trắng thừa ở hai đầu.
6. **Tách từ.** Hàm `nltk.word_tokenize` chuyển chuỗi thành danh sách token.
7. **Loại stopword tiếng Anh.** Các từ chức năng phổ biến trong danh sách NLTK được bỏ để giảm số chiều và hạn chế BoW bị chi phối bởi từ xuất hiện ở mọi lớp.
8. **Lemmatization.** `WordNetLemmatizer` đưa các biến thể như *bears* về *bear*. Do không gắn nhãn từ loại, lemmatizer mặc định thiên về danh từ và có thể không chuẩn hóa triệt để một số động từ.
9. **Ghép lại văn bản.** Các token còn lại được nối bằng một dấu cách và lưu trong cột `cleaned_text`.

Ví dụ dưới đây lấy từ mẫu đầu tiên của tập huấn luyện:

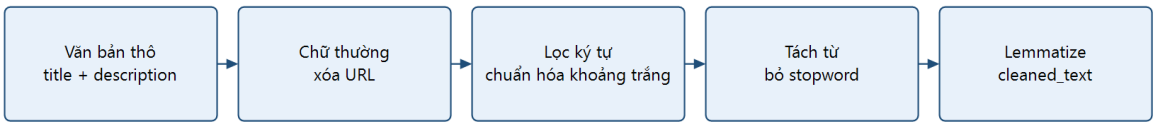
Gốc

Wall St. Bears Claw Back Into the Black (Reuters) Reuters – Short-sellers, Wall Street's dwindling\band of ultra-cynics, are seeing green again.

Sau tiền xử lý

```
wall st bear claw back black reuters reuters shortsellers wall
street dwindlingband ultracynics seeing green
```

Kết quả cho thấy quá trình chữ thường hóa, loại stopword và chuẩn hóa *bears* thành *bear* đã được thực hiện. Token `dwindlingband` cũng cho thấy một nhiễu còn lại: chuỗi escape từ dữ liệu thu thập bị dính vào từ đứng trước. Nhóm có thể cải thiện bước này bằng quy tắc xử lý chuỗi escape trước khi lọc ký tự.



Hình 6. Chuỗi tiền xử lý văn bản của pipeline học máy truyền thống.

Đầu ra của giai đoạn này gồm văn bản đã làm sạch, các ma trận BoW/TF–IDF cho train/validation/test và các vector nhãn tương ứng. Bộ vector hóa chỉ được `fit` trên phần train (102.000 mẫu). Các tập validation và test chỉ được `transform`. Nếu nhóm dùng kiểm định chéo, bộ vector hóa phải được học lại trong từng fold để tránh rò rỉ dữ liệu.

b. Trích xuất đặc trưng

b.1. Bag-of-Words

Với từ vựng V gồm tối đa 5.000 từ thường gặp nhất trên phần train, Count Vectorizer biểu diễn văn bản d_i bằng số lần xuất hiện của mỗi từ:

$$x_{i,j} = c(v_j, d_i), \qquad j = 1, \ldots, |V|. \tag{3}$$

Biểu diễn BoW đơn giản, dễ diễn giải và phù hợp với Multinomial Naive Bayes. Tuy nhiên, biểu diễn này không giữ thứ tự từ, không chuẩn hóa độ dài văn bản và có thể để các từ phổ biến như *reuters*, *said*, *year* lấn át từ khóa phân biệt.

b.2. TF–IDF

TF–IDF (Term Frequency–Inverse Document Frequency) đánh giá mức độ quan trọng của một từ dựa trên số lần xuất hiện của từ đó trong văn bản và độ phổ biến của từ trong toàn bộ phần train [1; 2]. Trong báo cáo này, vectorizer của scikit-learn dùng công thức IDF có làm trơn:

$$\mathrm{IDF}(t) = \ln\left(\frac{1+N}{1+\mathrm{DF}(t)}\right) + 1, \tag{4}$$

$$\text{TF-IDF}(t,d) = \mathrm{TF}(t,d)\,\mathrm{IDF}(t), \tag{5}$$

trong đó $N = 102.000$ là số văn bản dùng để `fit` vectorizer (phần train) và $\mathrm{DF}(t)$ là số văn bản train có chứa từ t. Sau khi tính TF–IDF, mỗi vector văn bản được chuẩn hóa theo chuẩn L_2. Vì vậy, từ xuất hiện trong hầu hết văn bản có IDF thấp, còn những từ mang tính chủ đề hơn, chẳng hạn *olympic*, *nasdaq*, *nasa* hoặc *iraq*, thường có IDF cao hơn.

Ở bước trích xuất đặc trưng, cả hai vectorizer đều dùng unigram và giới hạn từ vựng ở `max_features=5000`. Các tham số còn lại được giữ theo mặc định của scikit-learn. Vì văn bản đã được chuyển về chữ thường trong bước tiền xử lý, thiết lập `lowercase` mặc định của vectorizer không làm phát sinh thêm biến thể hoa/thường. Giới hạn 5.000 đặc trưng giúp cân bằng độ phủ từ vựng với thời gian huấn luyện, đồng thời hạn chế việc mô hình phụ thuộc vào các từ quá hiếm. Module `experiments/feature_selection.py` hỗ trợ các phương pháp chọn đặc trưng như kiểm định Chi-bình phương (χ^2), Mutual Information và `SelectFromModel`, nhưng các thí nghiệm chính không áp dụng bước này. Nhờ đó, việc so sánh tập trung vào hiệu quả của BoW và TF–IDF.

Bảng 4. Kích thước các ma trận đặc trưng thực nghiệm

Biểu diễn	Train	Validation	Test
TF–IDF	102.000×5.000	18.000×5.000	7.600×5.000
BoW	102.000×5.000	18.000×5.000	7.600×5.000

Các ma trận được lưu ở dạng thưa trong `data/processed/`. Hai bộ vector hóa đã khớp được lưu tại `models/tfidf_vectorizer.pkl` và `models/count_vectorizer.pkl` để tái sử dụng khi suy luận.

c. Huấn luyện và đánh giá mô hình

c.1. Thiết kế thí nghiệm

Nhóm huấn luyện đầy đủ sáu tổ hợp tạo bởi hai cách biểu diễn $\{\text{BoW}, \text{TF–IDF}\}$ và ba mô hình $\{\text{Multinomial NB}, \text{Logistic Regression}, \text{Linear SVM}\}$. Sáu tổ hợp được huấn luyện trên 102.000 mẫu train và được so sánh trên 18.000 mẫu validation. Sau đó, mô hình tốt nhất được đánh giá một lần trên 7.600 mẫu test. Tham số `random_state=42` được cố định khi thuật toán hỗ trợ tham số này. Thiết kế thí nghiệm cho phép so sánh thuật toán khi cố định đặc trưng, đồng thời so sánh đặc trưng khi cố định thuật toán.

Nhóm không dùng Kernel SVM vì chi phí tính toán cao trên khoảng 100.000 mẫu và 5.000 chiều. Các mô hình Random Forest, Decision Tree và Gradient Boosting có sẵn trong `TraditionalMLModels` nhưng không thuộc thí nghiệm chính, vì chúng thường kém phù hợp hơn mô hình tuyến tính trên dữ liệu văn bản rất thưa. SVM tuyến tính dựa trên nguyên lý cực đại hóa lề phân lớp [4].

c.2. Cơ sở toán học các mô hình và thiết lập siêu tham số

Nhóm lựa chọn ba mô hình học máy truyền thống đại diện cho các trường phái khác nhau: mô hình cơ sở xác suất (Multinomial Naive Bayes), mô hình phân loại tuyến tính xác suất (Logistic Regression), và mô hình tối ưu hóa biên lề cực đại (Linear SVM). Cả ba mô hình đều được xây dựng để xử lý dữ liệu ma trận thưa nhiều chiều với chi phí tính toán thấp.

1. **Multinomial Naive Bayes (MNB):** Mô hình dựa trên định lý xác suất Bayes với

giả thiết độc lập có điều kiện giữa các đặc trưng từ vựng:

$$P(Y = k \mid \mathbf{x}) = \frac{P(Y = k)P(\mathbf{x} \mid Y = k)}{P(\mathbf{x})} \propto P(Y = k) \prod_{j=1}^{|V|} P(w_j \mid Y = k)^{x_j}, \tag{6}$$

trong đó x_j là số lần xuất hiện (hoặc trọng số TF–IDF) của từ vựng w_j, và $|V| = 5.000$ là kích thước từ vựng. Xác suất hợp lý $P(w_j \mid Y = k) = \theta_{k,j}$ được ước lượng bằng kỹ thuật làm trơn Laplace (Laplace smoothing với $\alpha = 1{,}0$):

$$\hat{\theta}_{k,j} = \frac{N_{k,j} + \alpha}{N_k + \alpha|V|}, \tag{7}$$

với $N_{k,j}$ là tổng tần suất từ w_j trong lớp k, và $N_k = \sum_{j=1}^{|V|} N_{k,j}$. Để tránh hiện tượng tràn số dưới (underflow), quy tắc quyết định lớp được tính trong không gian log:

$$\hat{y} = \arg \max_{k \in \{1,2,3,4\}} \left[\ln P(Y = k) + \sum_{j=1}^{|V|} x_j \ln \hat{\theta}_{k,j} \right]. \tag{8}$$

2. **Logistic Regression (Softmax Regression):** Với bài toán phân loại bốn chủ đề ($K = 4$), mô hình xấp xỉ phân phối xác suất hậu nghiệm thông qua hàm kích hoạt Softmax:

$$P(Y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^T \mathbf{x} + b_k)}{\sum_{c=1}^{K} \exp(\mathbf{w}_c^T \mathbf{x} + b_c)}, \tag{9}$$

trong đó $\mathbf{w}_k \in \mathbb{R}^{|V|}$ là vector trọng số và $b_k \in \mathbb{R}$ là hệ số điều chỉnh (bias) cho lớp k. Quá trình huấn luyện giải bài toán tối thiểu hóa hàm mất mát Cross-Entropy kết hợp điều chuẩn L_2:

$$\min_{\mathbf{W},\mathbf{b}} \left\{ -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} \mathbb{I}(y_i = k) \ln P(Y = k \mid \mathbf{x}_i) + \frac{1}{2C} \sum_{k=1}^{K} \|\mathbf{w}_k\|_2^2 \right\}, \tag{10}$$

với $\mathbb{I}(\cdot)$ là hàm chỉ thị. Siêu tham số phạt nghịch đảo $C = 1{,}0$ kiểm soát độ mạnh của điều chuẩn, hạn chế hiện tượng quá khớp (overfitting) trên các từ hiếm. Nhóm cấu hình `max_iter=1000` với bộ giải mặc định `lbfgs`. Đáng chú ý, đối với bài toán Softmax đa lớp tối ưu hóa đồng thời hàm mất mát Cross-Entropy toàn cục, thuật toán L-BFGS chạy đơn tiến trình và bỏ qua tham số `n_jobs` (scikit-learn chỉ phân

luồng đa xử lý qua `n_jobs` khi áp dụng chiến lược One-vs-Rest độc lập); tuy vậy, quá trình tính toán vẫn được tăng tốc nhờ các phép toán ma trận của thư viện BLAS.

3. **Linear Support Vector Machine (Linear SVM):** Triển khai qua `LinearSVC` dựa trên thư viện LibLinear theo chiến lược One-vs-Rest (OvR). Trong scikit-learn, `LinearSVC` mặc định áp dụng hàm mất mát Hinge bậc hai (Squared Hinge Loss, `loss='squared_hinge'`) kết hợp điều chuẩn L_2 (`penalty='l2'`), giải bài toán tối ưu hóa lề mềm:

$$\min_{\mathbf{w}_k, b_k, \boldsymbol{\xi}} \left\{ \frac{1}{2}\|\mathbf{w}_k\|_2^2 + C\sum_{i=1}^{N} \xi_i^2 \right\} \quad \text{s.t.} \quad y_{i,k}(\mathbf{w}_k^T \mathbf{x}_i + b_k) \geq 1 - \xi_i, \quad \xi_i \geq 0, \tag{11}$$

tương đương với dạng hàm mục tiêu không ràng buộc trực tiếp:

$$\min_{\mathbf{w}_k, b_k} \left\{ \frac{1}{2}\|\mathbf{w}_k\|_2^2 + C\sum_{i=1}^{N} \max\left(0, 1 - y_{i,k}(\mathbf{w}_k^T \mathbf{x}_i + b_k)\right)^2 \right\}, \tag{12}$$

trong đó $y_{i,k} \in \{+1, -1\}$ (gán $+1$ nếu mẫu i thuộc lớp k, ngược lại gán -1), và ξ_i là biến bù (slack variable) đo mức vi phạm lề. Việc sử dụng số hạng phạt bậc hai ξ_i^2 (Hinge bậc 2) giúp hàm mục tiêu khả vi liên tục, cho phép thuật toán Coordinate Descent trong LibLinear hội tụ nhanh và ổn định trên dữ liệu văn bản lớn nhiều chiều. Hàm quyết định đa lớp là:

$$\hat{y} = \arg\max_{k \in \{1,2,3,4\}} (\mathbf{w}_k^T \mathbf{x} + b_k). \tag{13}$$

Siêu tham số $C = 1{,}0$ và `max_iter=1000` mang lại sự cân bằng tối ưu giữa việc mở rộng lề và giới hạn sai số phân lớp.

Các mô hình được huấn luyện bằng các cấu hình tham số mặc định chuẩn của Machine Learning ($C = 1{,}0$ đối với Linear SVM và Logistic Regression, $\alpha = 1{,}0$ với làm trơn Laplace ở Multinomial Naive Bayes). Việc giữ cố định các siêu tham số tiêu chuẩn này nhằm so sánh công bằng, khách quan hiệu năng bản chất giữa các thuật toán trên cùng hai không gian đặc trưng BoW và TF–IDF.

c.3. Độ đo đánh giá và cơ sở lựa chọn

Do bộ dữ liệu AG News có phân phối hoàn toàn cân bằng (**25%** mỗi lớp trên cả 120.000 mẫu train và 7.600 mẫu test), độ chính xác tổng thể (Accuracy) là độ đo phản ánh trung thực năng lực của mô hình:

$$\text{Accuracy} = \frac{\sum_{k=1}^{K} \text{TP}_k}{N_{\text{total}}}, \tag{14}$$

trong đó TP_k là số lượng mẫu dự đoán đúng của lớp k, và N_{total} là tổng số mẫu đánh giá.

Tuy nhiên, Accuracy chỉ cung cấp bức tranh tổng quát mà không phản ánh được mức độ sai sót cục bộ giữa từng chủ đề tin tức. Do đó, nhóm sử dụng thêm Precision, Recall và F1-Score theo từng lớp:

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \tag{15}$$

$$\text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}, \tag{16}$$

$$\text{F1}_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}, \tag{17}$$

với FP_k là số mẫu lớp khác bị dự đoán nhầm thành lớp k (False Positive), và FN_k là số mẫu thật của lớp k bị gán sai nhãn (False Negative).

Để đánh giá tổng hợp trên toàn bộ $K = 4$ chủ đề, nhóm sử dụng chỉ số Macro-F1:

$$\text{Macro-F1} = \frac{1}{K} \sum_{k=1}^{K} \text{F1}_k. \tag{18}$$

Chỉ số Macro-F1 gán trọng số bình đẳng $\frac{1}{4}$ cho từng lớp, ngăn ngừa trường hợp mô hình tối ưu thiên vị cho một lớp riêng biệt mà làm suy giảm chất lượng phân loại ở các lớp còn lại. Đại lượng Weighted-F1 được định nghĩa là $\sum_{k=1}^{K} \frac{N_k}{N_{\text{total}}} \text{F1}_k$; vì $N_k / N_{\text{total}} = 0{,}25$, giá trị Weighted-F1 thực nghiệm luôn tiệm cận Macro-F1, đóng vai trò kiểm chứng chéo tính nhất quán toán học.

Bên cạnh các chỉ số phân lớp, nhóm ghi nhận thời gian huấn luyện (*train time*) và thời gian suy luận (*predict time*) trong hàm `train_and_evaluate()` thuộc module `src/evaluation.py` để đo lường độ trễ tính toán. Ma trận nhầm lẫn (Confusion Matrix)

được trực quan hóa nhằm chỉ ra rõ các cặp lớp có sự giao thoa ngữ nghĩa cao như Business–Sci/Tech.

c.4. Kết quả trên tập validation

Bảng 5. Kết quả sáu tổ hợp đặc trưng và mô hình trên 18.000 mẫu validation

Mô hình	Đặc trưng	Acc.	Prec. macro	Rec. macro	F1 macro	F1 weighted	Train (s)	Predict (s)
Naive Bayes	BoW	0,8987	0,8983	0,8987	0,8984	0,8984	0,02	0,003
Logistic Regression	BoW	0,9028	0,9026	0,9028	0,9026	0,9026	1,90	0,003
Linear SVM	BoW	0,9040	0,9039	0,9040	0,9038	0,9038	7,17	0,002
Naive Bayes	TF–IDF	0,8990	0,8987	0,8990	0,8987	0,8987	0,01	0,002
Logistic Regression	TF–IDF	0,9127	0,9126	0,9127	0,9125	0,9125	1,89	0,001
Linear SVM	TF–IDF	**0,9138**	**0,9137**	**0,9138**	**0,9136**	**0,9136**	**2,79**	0,003

Ghi chú: Số liệu lấy từ `results/comparison.csv` (notebook 03). Mô hình tốt nhất sau đó được đánh giá trên tập test ở Mục e.

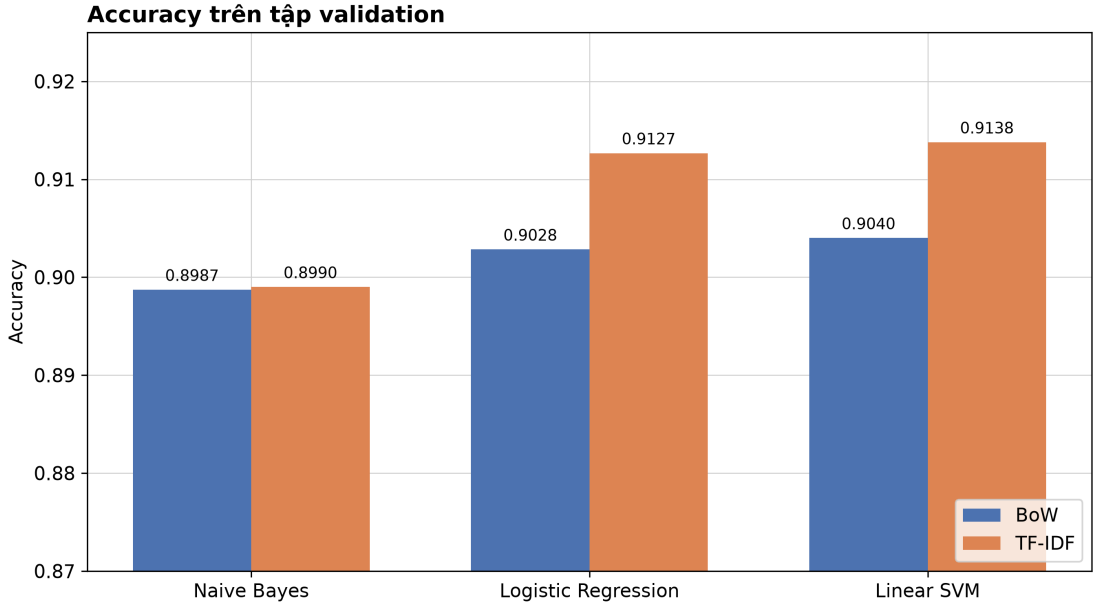
Mọi tổ hợp đều đạt từ khoảng 89,9% accuracy và F1 trở lên. Các kết quả này cao hơn rõ rệt đường cơ sở dự đoán ngẫu nhiên hoặc luôn dự đoán một lớp (**25%**). Accuracy, macro-F1 và weighted-F1 chênh nhau dưới 0,002 ở mọi cấu hình. Điều này phù hợp với phân bố nhãn cân bằng. Mô hình tốt nhất trên tập validation là Linear SVM kết hợp TF–IDF. Cấu hình này đạt accuracy 91,38% và macro-F1 91,36%, với thời gian huấn luyện khoảng 2,79 giây.

d. So sánh trực quan và phân tích kết quả

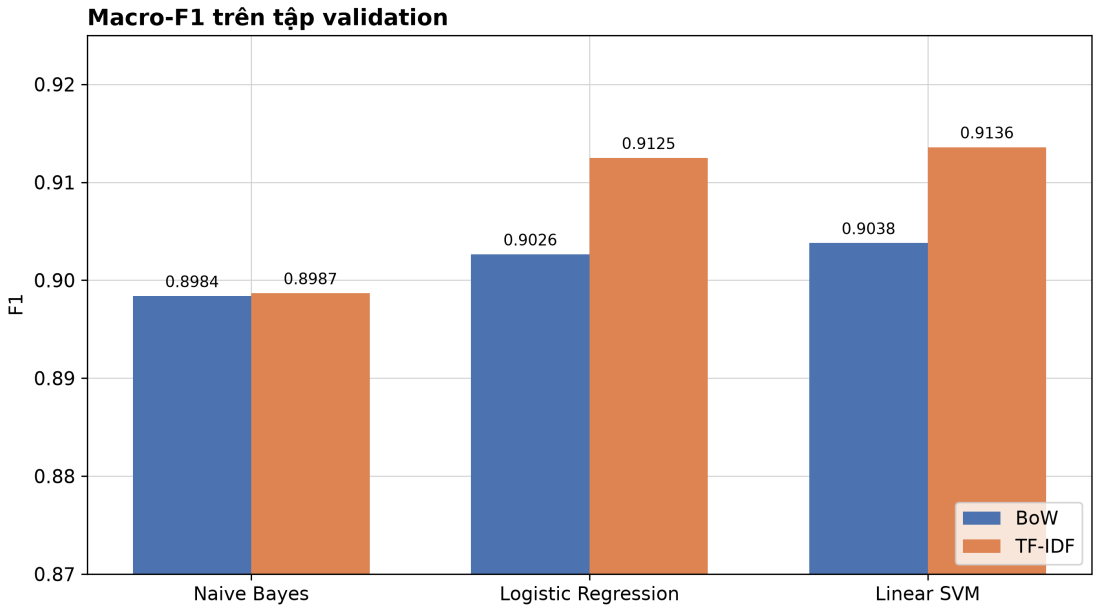
d.1. So sánh theo thuật toán

Trên BoW, thứ tự accuracy là Linear SVM (0,9040), Logistic Regression (0,9028) và Naive Bayes (0,8987). Trên TF–IDF, thứ tự không đổi với các giá trị accuracy tương ứng 0,9138; 0,9127 và 0,8990. Linear SVM đứng đầu ở cả hai cách biểu diễn. Kết quả này phù hợp với đặc điểm dữ liệu văn bản thưa, nhiều chiều và thường gần phân tách tuyến tính.

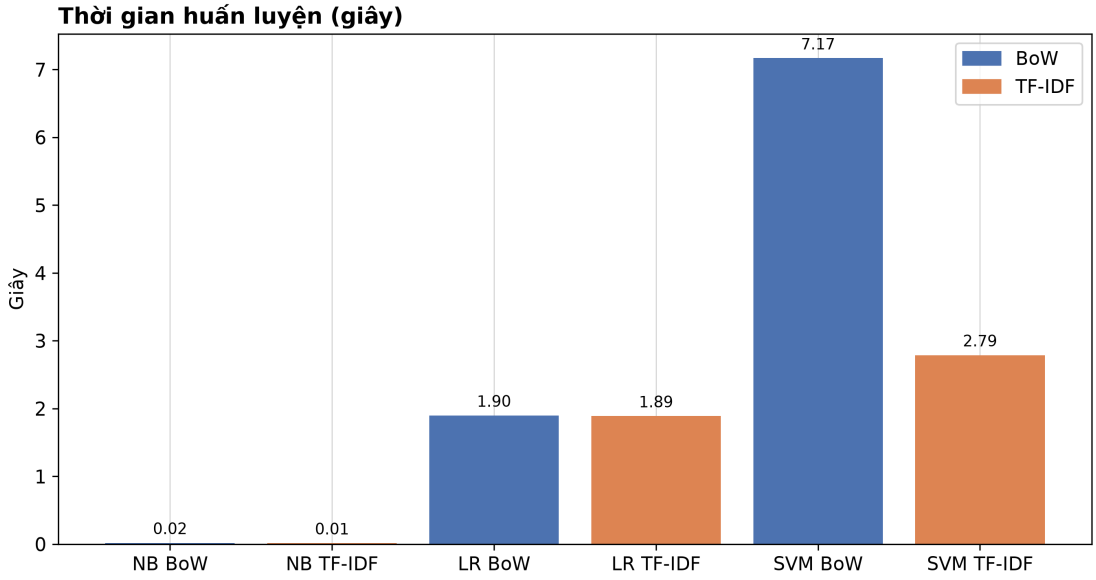
Logistic Regression tiến sát SVM khi dùng TF–IDF và chỉ kém khoảng 0,11 điểm phần trăm accuracy. Naive Bayes gần như không cải thiện khi đổi BoW sang TF–IDF, vì trọng số TF–IDF không còn tuân theo diễn giải đếm multinomial nguyên thủy.



Hình 7. Accuracy của ba thuật toán trên BoW và TF–IDF (validation). Trục tung được thu hẹp quanh vùng 0,87–0,92 để thấy chênh lệch giữa các cấu hình.



Hình 8. Macro-F1 của cùng sáu cấu hình trên tập validation. Thứ hạng gần như trùng với accuracy vì bốn lớp có cùng số mẫu.



Hình 9. Thời gian huấn luyện trên tập validation. Naive Bayes gần như tức thời. Linear SVM trên BoW là cấu hình chậm nhất; cùng thuật toán trên TF–IDF nhanh hơn rõ rệt.

d.2. So sánh BoW và TF–IDF

Trung bình trên ba mô hình, BoW đạt accuracy **0,9018** và macro-F1 0,9016. TF–IDF đạt accuracy **0,9085** và macro-F1 0,9083. Như vậy, TF–IDF cao hơn khoảng **0,67** điểm phần trăm accuracy. Logistic Regression hưởng lợi nhiều nhất (+**0, 99** điểm). Linear SVM tăng khoảng 0,98 điểm. Naive Bayes chỉ tăng rất nhẹ 0,03 điểm.

TF–IDF cũng giúp Linear SVM giảm thời gian huấn luyện từ **7,17** xuống **2,79** giây trong lần đo này. Kết quả thời gian phụ thuộc môi trường và phiên bản thư viện, nhưng hợp lý vì đặc trưng TF–IDF đã được scale và chuẩn hóa tốt hơn giá trị đếm thô.

d.3. Thời gian và lỗi định tính

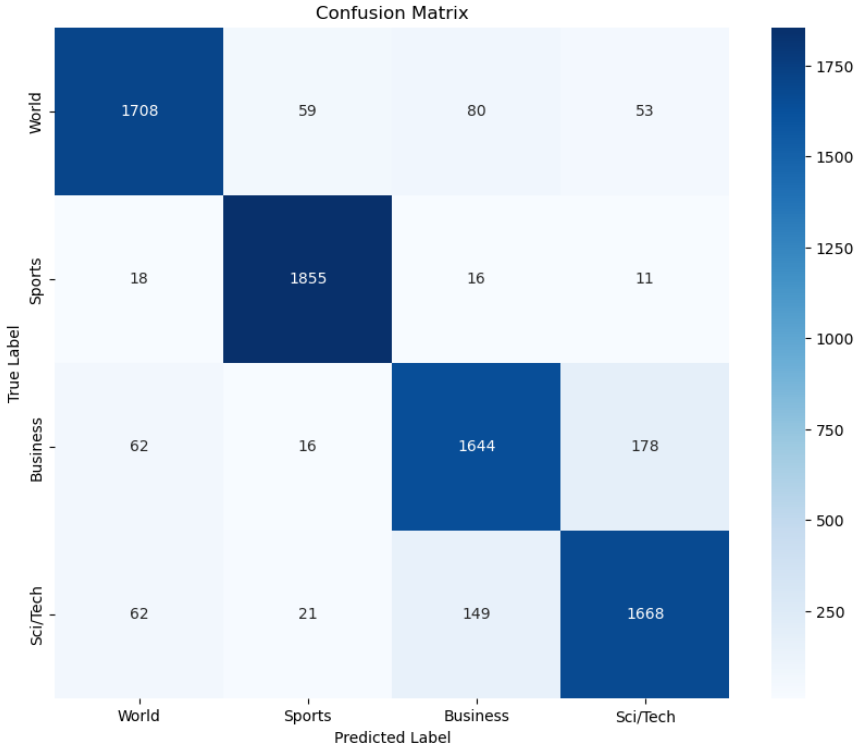
Hình 9 cho thấy Naive Bayes chỉ cần khoảng **0,02** giây để huấn luyện, nên phù hợp làm đường cơ sở và kiểm thử pipeline. Logistic Regression cần khoảng hai giây. SVM–BoW là cấu hình chậm nhất. Ngược lại, SVM–TF–IDF vừa chính xác nhất vừa nhanh hơn đáng kể so với SVM–BoW. Chênh lệch khoảng **1,51** điểm accuracy giữa NB–BoW và mô hình tốt nhất tương ứng xấp xỉ **272** dự đoán đúng thêm trên 18.000 mẫu validation.

Lớp Sports có từ vựng riêng, trong khi Business và Sci/Tech chia sẻ nhiều từ về công ty và sản phẩm. Mức nhầm lẫn cụ thể được đo lại ở Mục e trên cấu hình Linear SVM–TF–IDF khi đánh giá tập test.

Giới hạn chính của unigram là không mã hóa thứ tự từ và ngữ cảnh đa nghĩa. Đây cũng là hướng mở rộng đã nêu ở phần kết luận, dù pipeline tuyến tính đã đạt khoảng 91,4% trên validation và 90,5% trên test.

e. Đánh giá tập test và phân tích lỗi theo lớp

Bảng 5 ghi nhận số liệu so sánh từ `results/comparison.csv` trên tập validation (được thực thi trong `notebooks/03_model_training.ipynb`). Sau khi xác định mô hình chiến thắng là Linear SVM kết hợp biểu diễn đặc trưng TF–IDF ($C = 1{,}0$), nhóm tiến hành quy trình đánh giá độc lập trên toàn bộ 7.600 mẫu của tập kiểm tra thông qua `notebooks/04_final_test_evaluation.ipynb`. Trên tập test độc lập, cấu hình này đạt accuracy 0,9051 và macro-F1 0,9049. Mức sụt giảm hiệu năng so với tập validation chỉ khoảng 0,87 điểm phần trăm. Độ chênh lệch tối thiểu này chứng minh mô hình có khả năng khái quát hóa vững chắc, hoàn toàn không bị rò rỉ thông tin (data leakage) hay quá khớp (overfitting) vào tập dữ liệu phát triển.

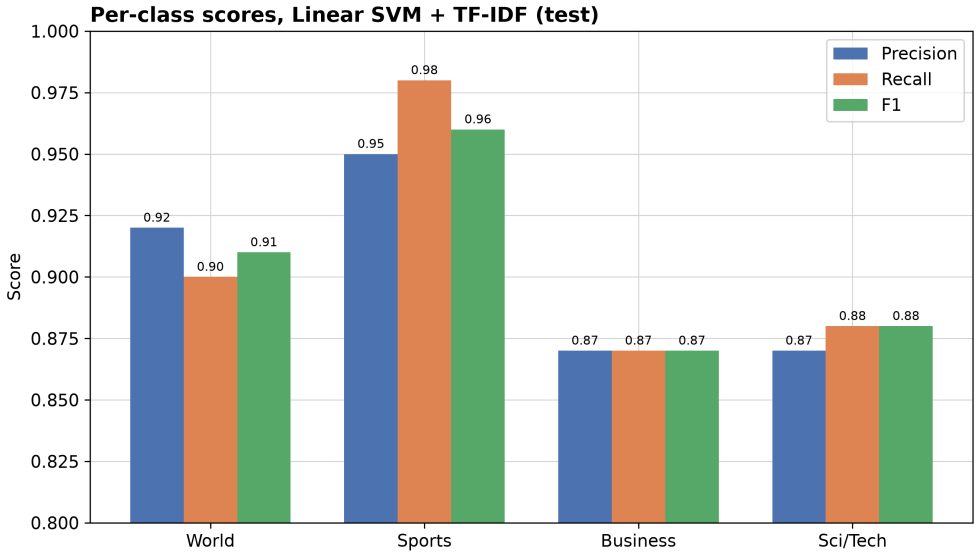


Hình 10. Ma trận nhầm lẫn của Linear SVM + TF–IDF trên 7.600 mẫu test. Hàng là nhãn đúng, cột là nhãn dự đoán.

Bảng 6. Chỉ số theo lớp của Linear SVM + TF–IDF trên tập test

Lớp	Precision	Recall	F1	Support
World	0,92	0,90	0,91	1.900
Sports	0,95	0,98	**0,96**	1.900
Business	0,87	0,87	**0,87**	1.900
Sci/Tech	0,87	0,88	**0,88**	1.900
Macro	0,9049	0,9051	0,9049	7.600

Ghi chú: Precision, recall và F1 từng lớp được làm tròn 2 chữ số theo classification report; hàng Macro giữ 4 chữ số từ các độ đo tổng hợp.



Hình 11. Precision, recall và F1 theo lớp trên tập test (Linear SVM + TF–IDF). Trục tung bắt đầu ở 0,80 để thấy Sports tách khỏi hai lớp kinh doanh và công nghệ.

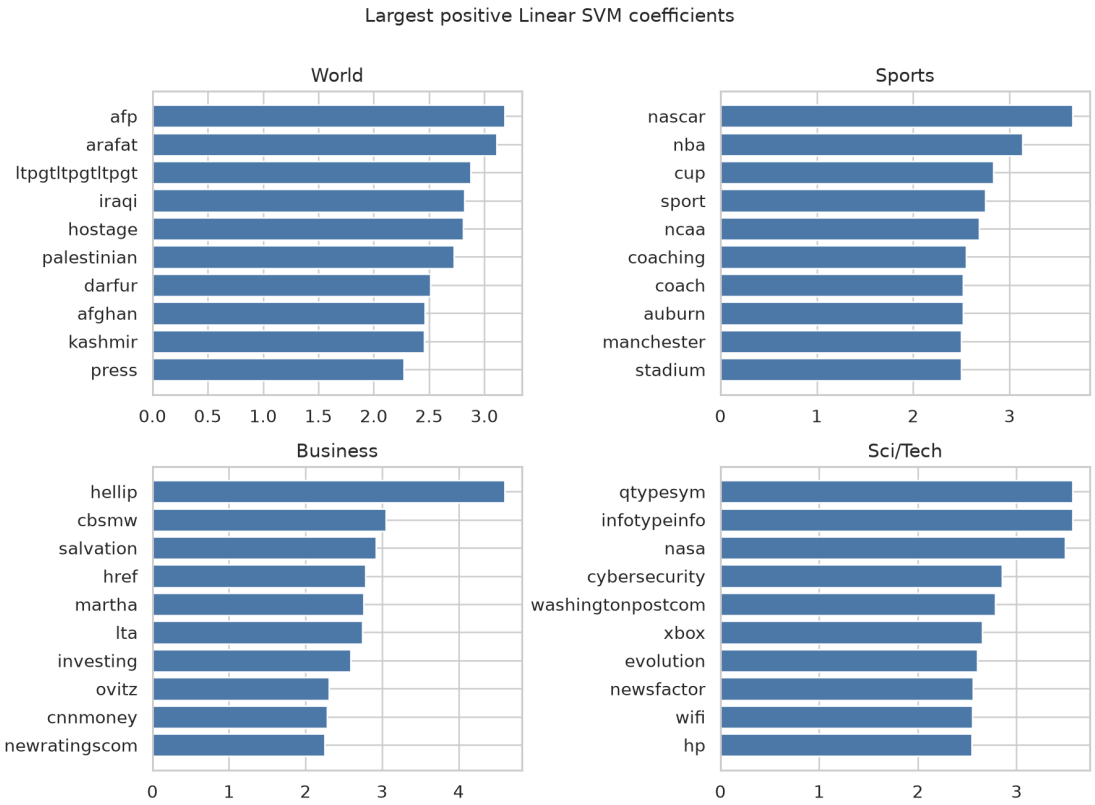
Lớp Sports đạt recall khoảng 0,98 và F1 khoảng 0,96. Lớp World cũng cao với F1 khoảng 0,91. Hai lớp Business và Sci/Tech thấp hơn, với F1 lần lượt khoảng 0,87 và 0,88. Ma trận nhầm lẫn cho thấy lỗi tập trung chủ yếu ở cặp Business–Sci/Tech. Vì vậy, accuracy khoảng 0,91 không có nghĩa bốn lớp cùng khó. Phần khó chủ yếu nằm ở ranh giới giữa tin kinh doanh và tin công nghệ.

Ba mẫu sau minh họa ranh giới đó và không phải là lỗi ngẫu nhiên:

- Nhãn thật là Sci/Tech, mô hình dự đoán Business: *IBM to hire even more new workers... biggest headcount since 1991.* Nội dung nói về tuyển dụng của một hãng máy tính.
- Nhãn thật là Sci/Tech, mô hình dự đoán Business: *Dell exits low-end China consumer PC market.* Nội dung vừa liên quan sản phẩm công nghệ, vừa liên quan quyết định thị trường.
- Nhãn thật là Business, mô hình dự đoán Sci/Tech: *Intel to delay product aimed for high-definition TVs.* Nội dung nói về việc hãng bán dẫn lùi lịch ra mắt một con chip.

Các hệ số dương lớn nhất của SVM cho thấy mô hình học được từ đúng chủ đề, nhưng đồng thời cũng bị kéo bởi nhiễu thu thập dữ liệu. Lớp Sports gắn với *nascar*, *nba*, *coach*, *ncaa*. Lớp World gắn với *iraqi*, *palestinian*, *darfur*, *afghan*, nhưng cũng xuất hiện *afp* và

một token dính từ thẻ HTML. Lớp Business có *investing* cạnh *hellip* và *href*. Lớp Sci/Tech có *nasa*, *xbox*, *wifi* cạnh các mảnh XML như *qtypesym*. Vì SVM tuyến tính cộng trọng số từng từ, việc làm sạch thực thể HTML trước bước lọc ký tự là cải tiến tiền xử lý có căn cứ, chứ không chỉ dựa trên cảm tính.



Hình 12. Mười hệ số dương lớn nhất của Linear SVM trên từng lớp. Trục hoành là trọng số trong không gian TF–IDF.

f. Thực nghiệm lựa chọn đặc trưng (Feature Selection)

Để khảo sát xem việc giảm số chiều có giúp loại bỏ nhiễu và tăng tốc độ mô hình mà không làm giảm độ chính xác hay không, nhóm đã triển khai module `experiments/feature_selection.py` với ba phương pháp chọn thuộc tính:

1. **Kiểm định Chi-bình phương (χ^2):** Đánh giá mức độ phụ thuộc thống kê giữa sự xuất hiện của từ t và từng lớp chủ đề k:

$$\chi^2(t,k) = \frac{N(AD - BC)^2}{(A+B)(C+D)(A+C)(B+D)}, \tag{19}$$

trong đó A là số văn bản lớp k chứa t; B là số văn bản ngoài lớp k chứa t; C là số văn bản lớp k không chứa t; D là số văn bản ngoài lớp k không chứa t; và $N = A + B + C + D$. Điểm đặc trưng tổng hợp là $\chi^2_{\max}(t) = \max_k \chi^2(t, k)$.

2. **Thông tin tương hỗ (Mutual Information):** Đo lượng thông tin thu được về nhãn chủ đề Y khi biết sự hiện diện hay vắng mặt của từ t. Gọi $X_t \in \{0, 1\}$ là biến ngẫu nhiên nhị phân biểu thị sự xuất hiện ($x = 1$) hoặc vắng mặt ($x = 0$) của từ t trong văn bản:

$$I(X_t; Y) = \sum_{x \in \{0,1\}} \sum_{k=1}^{K} P(X_t = x, Y = k) \log_2 \frac{P(X_t = x, Y = k)}{P(X_t = x) P(Y = k)}, \tag{20}$$

trong đó $P(X_t = x, Y = k)$ là phân phối xác suất đồng thời, $P(X_t = x)$ và $P(Y = k)$ là các xác suất biên.

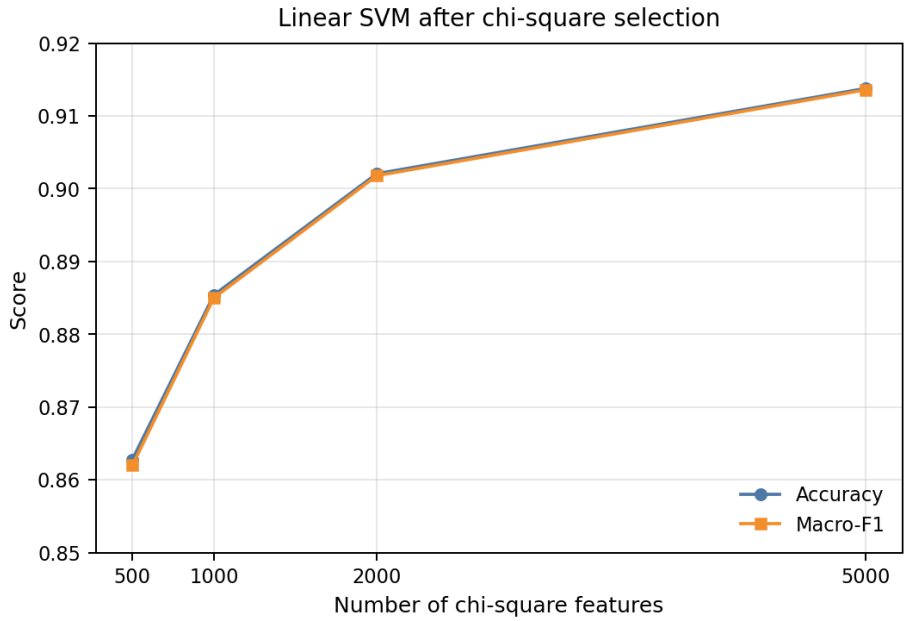
3. **Mô hình cây (Tree-based):** Sử dụng kỹ thuật `SelectFromModel` kết hợp ensemble `RandomForest` đo lường mức giảm độ tạp chất Gini (Mean Decrease Impurity) để xếp hạng các đặc trưng quan trọng nhất.

Thực nghiệm trên phương pháp Chi-bình phương được tiến hành bằng cách giữ lại top k từ phân biệt nhất từ không gian TF–IDF ban đầu, sau đó huấn luyện mô hình Linear SVM ($C = 1{,}0$). Bảng 7 và Hình 13 trình bày kết quả khi quét $k \in \{500, 1000, 2000, 5000\}$.

Bảng 7. Hiệu năng Linear SVM sau khi chọn đặc trưng bằng Chi-bình phương

Số đặc trưng (k)	Accuracy	Macro-F1	Train (s)
500	0,8628	0,8620	1,47
1.000	0,8854	0,8850	1,71
2.000	0,9021	0,9018	2,04
5.000	**0,9138**	**0,9136**	2,82

Ghi chú: Điểm số tăng đơn điệu theo k. Số liệu lấy từ thực nghiệm trong `experiments/feature_selection.py`.



Hình 13. Accuracy và macro-F1 tăng khi giữ nhiều đặc trưng chi-bình phương hơn. Tại $k = 500$, cả hai độ đo còn khoảng 0,86.

Kết quả chứng minh rằng với bài toán phân loại tin tức đa lớp AG News, không gian 5.000 từ vựng ban đầu chưa bị hiện tượng bão hòa hay quá nhiều chiều dẫn đến quá khớp. Khi cắt giảm k xuống dưới 2.000 từ (đặc biệt tại $k = 500$, accuracy sụt giảm xuống 0,8628), hiệu năng mô hình giảm sút rõ rệt do việc lược bỏ quá mức đã tước đi các từ khóa chuyên biệt theo từng chủ đề hẹp (như tên riêng địa chính trị, tên vận động viên, hoặc thuật ngữ công nghệ mới nổi). Do đó, cấu hình giữ nguyên 5.000 đặc trưng mang lại sự cân bằng tối ưu nhất giữa độ chính xác và chi phí bộ nhớ.

4. Tổ chức và quản lý mã nguồn của nhóm

Nhóm quản lý mã nguồn bằng Git tại https://github.com/Vhoamochii/ML_assignment. Logic dùng chung được đặt trong `src/`. Quy trình thí nghiệm nằm trong `notebooks/`. Các tinh chỉnh bổ sung nằm trong `experiments/`. Seed `random_state=42` được cố định để tái lập kết quả. Dữ liệu thô, ma trận đặc trưng và mô hình lớn không được commit vào kho mã. Liên kết tải nằm trong README và `models/model_links.txt`.

a. Cấu trúc thư mục

```
ML_assignment/
|-- data/
|   |-- raw/                          # Du lieu goc: train.csv (120k), test.csv
    (7.6k)
|   `-- processed/                    # Du lieu da tien xu ly va ma tran dac
    trung
|-- notebooks/                        # 4 Notebooks theo luong pipeline tuan tu
|   |-- 01_data_analysis.ipynb        # Phan tich va kham pha dac trung du lieu
    (EDA)
|   |-- 02_preprocessing.ipynb        # Lam sach van ban, trich xuat dac trung
    BoW & TF-IDF
|   |-- 03_model_training.ipynb       # Huan luyen 6 to hop mo hinh tren tap
    validation
|   `-- 04_final_test_evaluation.ipynb # Danh gia mo hinh tot nhat tren tap test
    doc lap
|-- src/                              # Ma nguon module hoa (Modular
    Architecture)
|   |-- data_loader.py                # Nap va phan chia tap du lieu
|   |-- preprocessing.py              # Tien xu ly van ban, tokenization,
    lemmatization
|   |-- evaluation.py                 # Tinh toan metrics va ve bieu do
    confusion matrix
|   |-- utils.py                      # Luu tru / nap mo hinh (.pkl) va tien ich
|   `-- models/                       # Dinh nghia cac thuat toan Machine
    Learning
|       |-- baseline.py               # Dummy classifier, Naive Bayes, Logistic
    Regression
|       `-- traditional_ml.py         # Linear SVM, SVC, Random Forest, Decision
     Tree
|-- experiments/                      # Thi nghiem lua chon dac trung
|   `-- feature_selection.py          # Lua chon dac trung (Chi2, Mutual Info,
    Tree-based)
|-- models/
|   `-- model_links.txt               # Lien ket luu tru cac mo hinh lon
|-- results/
|   |-- comparison.csv                # Bang tong hop ket qua tren tap
    validation
|   `-- figures/                      # Cac bieu do truc quan hoa va ma tran
    nham lan
|-- Dockerfile                        # Thiet lap moi truong Docker dong nhat
```

```
|-- requirements.txt                    # Danh sach goi phu thuoc Python
|-- README.md                           # Huong dan van hanh va tai lap ket qua
`-- .gitignore
```

b. Luồng chạy tái lập

Quy trình tái lập kết quả thực nghiệm được chuẩn hóa qua hai phương thức triển khai môi trường và một chuỗi thực thi tuần tự khép kín:

1. **Khởi tạo môi trường:**
   - *Phương thức 1 (Docker):* Xây dựng image và khởi chạy container với cổng 8888:

     `docker build -t ag-news-ml-project .`

     `docker run -p 8888:8888 -v ${PWD}:/app ag-news-ml-project`
   - *Phương thức 2 (Local pip):* Cài đặt các gói phụ thuộc qua `pip install -r requirements.txt` và tải từ điển NLTK (`punkt`, `stopwords`, `wordnet`).
2. **Chuẩn bị dữ liệu:** Tải bộ dữ liệu AG News từ Kaggle và đặt hai tệp `train.csv` (120.000 mẫu) và `test.csv` (7.600 mẫu) vào thư mục `data/raw/`.
3. **Khám phá dữ liệu (EDA):** Mở và thực thi `notebooks/01_data_analysis.ipynb` để thống kê phân phối lớp, độ dài văn bản và xuất biểu đồ vào `results/figures/`.
4. **Tiền xử lý và tạo đặc trưng:** Thực thi `notebooks/02_preprocessing.ipynb` để làm sạch văn bản, tách tập train/validation (102.000 / 18.000 mẫu), khớp vectorizer và lưu ma trận BoW, TF–IDF vào `data/processed/`.
5. **Huấn luyện mô hình:** Thực thi `notebooks/03_model_training.ipynb` để huấn luyện sáu tổ hợp mô hình, đánh giá trên 18.000 mẫu validation, lưu mô hình vào `models/` và ghi bảng `results/comparison.csv`.
6. **Đánh giá kiểm thử độc lập:** Thực thi `notebooks/04_final_test_evaluation.ipynb` để nạp mô hình tối ưu (`linear_svm_tfidf.pkl`), dự đoán một lần trên 7.600 mẫu test, vẽ ma trận nhầm lẫn và báo cáo theo lớp.
7. **Thực thi thí nghiệm mở rộng:** Chạy từ terminal tập lệnh `python experiments/feature_selection.py` để khảo sát ảnh hưởng của việc lựa chọn đặc trưng với Chi-bình phương, Mutual Information và Tree-based.

Thứ tự thực thi từ notebook 01 đến 04 cần được tuân thủ nghiêm ngặt, vì đầu ra của giai đoạn trước đóng vai trò dữ liệu đầu vào cho giai đoạn kế tiếp. Toàn bộ quá trình sử dụng chung một seed ngẫu nhiên `random_state=42` nhằm bảo đảm tính tiền định (determinism) và tính tái lập hoàn toàn (100% reproducibility).

5. Thảo luận

AG News phù hợp với bài toán phân loại văn bản vì bộ dữ liệu đã có nhãn sẵn, có tập kiểm tra tách biệt, bốn lớp cân bằng và độ dài văn bản vừa đủ để BoW cùng TF–IDF chạy được trên CPU. Khó khăn chính **không nằm ở mất cân bằng lớp**, mà nằm ở nhiễu từ quá trình thu thập dữ liệu, ở từ đa nghĩa, và ở sự chồng lấn từ vựng giữa Business với Sci/Tech khi cùng nói về công ty hay sản phẩm. Ma trận nhầm lẫn trên tập test củng cố nhận định này: **lớp Sports gần như được phân tách rõ**, trong khi lỗi còn lại tập trung chủ yếu ở cặp Business–Sci/Tech.

Vì mỗi lớp chiếm đúng 25%, accuracy là độ đo hợp lệ. Một lớp không thể làm tăng điểm chung chỉ nhờ có nhiều mẫu hơn. Báo cáo vẫn dùng thêm macro-F1 để kiểm tra xem có lớp nào bị hy sinh hay không. Trên sáu cấu hình validation, accuracy và macro-F1 lệch nhau dưới 0,002. Khi nhìn theo từng lớp trên tập test, khoảng cách xuất hiện rõ hơn: **lớp Sports đạt F1 khoảng** 0,96, còn **Business và Sci/Tech chỉ khoảng** 0,87–0,88. Nếu chỉ nhìn accuracy khoảng 0,91, người đọc dễ bỏ qua đúng chỗ mô hình còn yếu.

Ba thuật toán được so sánh trên cùng hai cách biểu diễn BoW và TF–IDF. Naive Bayes đóng vai trò đường cơ sở nhanh. Logistic Regression và Linear SVM là hai mô hình tuyến tính với cách điều chuẩn khác nhau. Kết quả cho thấy **Linear SVM kết hợp TF–IDF là cấu hình tốt nhất** (validation 0,9138; test 0,9051). Cấu hình này vừa cao về accuracy vừa nhanh hơn SVM trên BoW. Thí nghiệm bổ sung về lựa chọn đặc trưng bằng kiểm định chi-bình phương cho thấy việc cắt giảm bớt số chiều xuống dưới 5.000 từ không cải thiện được hiệu năng so với cấu hình gốc 5.000 chiều (0,9138). Kết quả này khẳng định không gian đặc trưng TF–IDF 5.000 chiều kết hợp nguyên lý cực đại hóa lề của Linear SVM đã đạt được sự cân bằng tối ưu giữa độ phủ từ vựng và chi phí tính toán.

Pipeline hiện tại còn bị giới hạn bởi biểu diễn unigram. Biểu diễn này không giữ thứ tự từ, khó xử lý từ đa nghĩa và câu diễn đạt khác nhau cùng một ý, đồng thời phụ thuộc vào chất lượng bước làm sạch. Phân tích hệ số của SVM cũng cho thấy nhiễu HTML/XML vẫn lọt vào từ vựng. Vì vậy, **cải thiện tiền xử lý có thể giảm lỗi mà chưa cần đổi**

thuật toán.

6. Kết luận và hướng phát triển

Báo cáo đã xây dựng một pipeline học máy có giám sát để phân loại tin tức AG News thành bốn chủ đề. Qua so sánh có kiểm soát giữa BoW/TF–IDF và các mô hình Naive Bayes, Logistic Regression, Linear SVM, nhóm rút ra một số nhận định mang tính học máy như sau.

Thứ nhất, **cách biểu diễn dữ liệu quan trọng không kém thuật toán**. Cùng một mô hình, TF–IDF thường tốt hơn BoW vì làm nổi bật từ đặc trưng chủ đề và giảm ảnh hưởng của từ xuất hiện ở mọi lớp. Thứ hai, trên không gian đặc trưng văn bản thưa và nhiều chiều, **mô hình tuyến tính đã đủ mạnh**. Linear SVM–TF–IDF là cấu hình tốt nhất trong các thí nghiệm. Kết quả này cho thấy biên quyết định tuyến tính phù hợp với túi từ. Thứ ba, **accuracy cao không đồng nghĩa mọi lớp đều dễ**. Lớp Sports tách khá rõ nhờ từ vựng riêng, trong khi Business và Sci/Tech dễ nhầm vì dùng chung vốn từ về công ty và sản phẩm. Thứ tư, không gian đặc trưng 5.000 từ vựng đã bao phủ tối ưu thông tin phân lớp; việc cắt giảm số chiều thông qua lựa chọn đặc trưng chi-bình phương làm giảm dần độ chính xác do tước đi các từ khóa chuyên biệt của từng chủ đề. Cuối cùng, nhiễu HTML/XML còn sót trong từ vựng cho thấy **chất lượng tiền xử lý quyết định trực tiếp độ sạch của không gian đặc trưng**. Nhóm có thể cải thiện mô hình trước khi đổi thuật toán.

Về hướng phát triển, nhóm nên ưu tiên **làm sạch dữ liệu và làm giàu biểu diễn**: loại thực thể HTML/XML trước khi lọc ký tự, rồi bổ sung bigram để giữ cụm từ mang nghĩa như *wall street* hay *interest rate*. Chỉ sau khi hai bước này ổn định, nhóm mới nên thử mô hình học sâu hoặc encoder tiền huấn luyện. Nếu vẫn dùng túi từ nhiễu thì mô hình phức tạp hơn chưa chắc khai thác thêm được tín hiệu ngữ nghĩa. Mọi thí nghiệm mở rộng cần giữ phép chia chính thức train/test, chỉ fit vectorizer trên phần train, dùng validation để chọn cấu hình, đánh giá test đúng một lần, và lưu kết quả vào file riêng để so sánh công bằng với đường cơ sở hiện tại.

Tài liệu tham khảo

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[3] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," in *AAAI Workshop on Learning for Text Categorization*, 1998.

[4] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.

[7] Scikit-learn developers, "Feature extraction: text," https://scikit-learn.org/stable/modules/feature_extraction.html.

[8] A. Anand, "AG News Classification Dataset," Kaggle, https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset.

[9] Bộ môn, "Machine Learning Project (CO3117) – Project Guide," Trường Đại học Bách khoa TP.HCM.